D.K.T.E Society's Textile and Engineering Institute, Ichalkaranji

(An Empowered Autonomous Institute Affiliated to Shivaji University, Kolhapur).

Department of Computer Science and Engineering(AI-ML)

Lab Manual
of
Deep Learning Lab

For

T.Y.CSE (AIML) B.Tech.

(2025-2026)

Prepared By: Mr. Deepak Dilip Mahajan,
Asst.Professor, Department of CSE(AIML),
DKTE Society's Textile & Engineering Institute, Rajwada, Ichalkaranji

Course Objective

1. Provide hands-on experience in implementing core deep learning models, including fully connected networks, CNNs, and sequential models using modern deep learning frameworks.

2. Enable students to analyze and optimize deep learning models using initialization strategies, regularization techniques, learning rate scheduling, and early stopping.

3. Expose students to modern deep learning architectures and training paradigms such as transfer learning, recurrent models, and gated architectures for real-world datasets.

4. Introduce Explainable AI (XAI) techniques to interpret and analyze black-box deep learning models, enhancing transparency and trustworthiness.

5. Equip students with tools to evaluate fairness, detect bias, and generate counterfactual explanations in AI models, promoting ethical and responsible AI development.

Expected Course Outcome

After completing all experiments, students will be able to: -

1. Implement and train deep learning models
2. Evaluate deep learning model performance
3. Apply transfer learning and sequential models to solve practical machine learning problems
4. Apply explainable AI techniques such as LIME, SHAP, and Grad-CAM to interpret predictions of deep neural networks.
5. Analyze and mitigate bias and fairness issues in AI systems using quantitative fairness metrics and counterfactual explanations.
6. Demonstrate ethical awareness and responsible AI practices by critically analyzing real-world AI case studies and proposing transparent, fair solutions.

Deep Learning Lab
List of Experiments
(2025-26)

Sr. No.	Experiment	Expected Date of Conduction
1.	Deep Learning Foundations & Optimization	2nd Week January
2.	Overfitting Analysis and Regularization Techniques	2nd Week January
3.	Learning Rate Scheduling and Early Stopping	3rd Week January
4.	CNN for Image Classification	4th Week January
	Lab Test 1	1st Week February
5.	Transfer Learning using Pre-trained CNN	2nd Week February
6.	Sequential Models – RNN, LSTM, GRU	3rd Week February
7.	Interpreting Tree-Based Models	4th Week February
8.	Model Explainability using LIME	1st Week March
	Lab Test 2	2nd Week March
9.	Model Explainability using SHAP	3rd Week March
10.	CNN Visualization using Grad-CAM	4th Week March
11.	Counterfactual Explanations & Fairness Analysis	1st Week April
12.	Explainability & Ethics Case Study	2nd Week April
13.	Lab Test 3	3rd Week April
14.	DL Model Presentation / Case Study	4th Week April

Prepared By: Mr. Deepak Mahajan	Approved By: Dr. S. K. Shirgave

PART A: CORE DEEP LEARNING IMPLEMENTATION LAB

(Exp. No.1 -6, Supports DL Theory concepts and provides base models for XAI)

Experiment 1: Deep Learning Foundations & Optimization

Mapped to: Deep Learning (Unit 1)

Objective: To analyze the impact of Weight Initialization strategies (Xavier vs. He) and Optimization algorithms (SGD vs. Adam) on training stability and convergence speed.

1. Problem Statement:

Training deep neural networks is difficult due to issues like Vanishing/Exploding Gradients. In this experiment, you will train a "Deep Perceptron" (Multi-Layer Perceptron) on the MNIST dataset to observe how different configurations affect training.

You will compare:

- Initialization: RandomNormal vs. GlorotUniform (Xavier) vs. HeNormal.
- Optimizers: SGD (Stochastic Gradient Descent) vs. Adam.

2. Prerequisites (Theory)

Vanishing Gradient: When gradients become too small during backpropagation, earlier layers stop learning. Proper initialization helps prevent this.

- Xavier Initialization (Glorot): Best for Sigmoid/Tanh activation functions.
- He Initialization: Best for ReLU activation functions.

Optimizers:

- SGD: Updates weights using a small batch; can be slow and oscillate.
- Adam: Adaptive learning rate method; usually converges faster and is more stable.

3. Procedure

Step 1: Environment Setup

- Open Google Colab / Jupyter Notebook.

- Import required libraries:
- TensorFlow / Keras
- NumPy
- Matplotlib
- Set random seeds to ensure reproducibility.

Step 2: Dataset Loading and Preprocessing

- Load the MNIST dataset using Keras.
- Split data into training and testing sets.
- Normalize pixel values to the range [0, 1].
- Flatten the 28×28 images into 784-dimensional vectors.
- Convert class labels into one-hot encoded vectors.

Step 3: Model Architecture Design

- Design a Deep Perceptron (MLP) with:
- Input layer (784 neurons)
- Two or more hidden layers (e.g., 128, 64 neurons)
- Output layer (10 neurons with Softmax activation)
- Use ReLU activation for hidden layers.
- Keep architecture constant across experiments to ensure fair comparison.

Step 4: Weight Initialization Experiments

- Train the same model using different weight initialization strategies:
- Random Normal Initialization
- Glorot Uniform (Xavier) Initialization
- He Normal Initialization
- For each initialization:
- Compile the model using a fixed optimizer.
- Train for a fixed number of epochs.
- Record training loss and accuracy.

Step 5: Optimizer Comparison

Using the best-performing initialization, compare: SGD Optimizer and Adam Optimizer

For each optimizer:

- Train the model for the same number of epochs.
- Use identical batch size and learning rate (where applicable).
- Record convergence speed and stability.

Step 6: Performance Visualization

- Plot Training Loss vs. Epochs.
- Plot Training Accuracy vs. Epochs.

Compare:

- Speed of convergence
- Smoothness of learning curves
- Final accuracy achieved

Step 7: Result Analysis

- Observe which initialization leads to:
- Faster convergence
- Stable gradient flow
- Compare SGD and Adam in terms of:
- Training stability
- Sensitivity to learning rate
- Identify the best configuration.

4. Expected Outcome (Learning Outcomes): After completing this experiment, students will be able to:

- Understand the vanishing and exploding gradient problems in deep networks.
- Analyze the role of weight initialization in training stability.
- Differentiate between Xavier and He initialization based on activation functions.
- Compare SGD and Adam optimizers in terms of convergence speed.
- Visualize and interpret loss and accuracy curves.
- Select appropriate initialization–optimizer combinations for deep learning models.

5. What Students Should Write in the Lab Record

1. Aim: To study the effect of different weight initialization strategies and optimization algorithms on the training stability and convergence speed of a deep neural network using the MNIST dataset.

2. Problem Statement: Training deep neural networks often suffers from vanishing or exploding gradients, which slow down or prevent convergence. This experiment evaluates how different weight initialization methods and optimizers affect the learning behavior of a Multi-Layer Perceptron.

3. Dataset Description

- Dataset Name: MNIST
- Number of Classes: 10
- Image Size: 28 × 28 grayscale
- Training Samples: 60,000
- Testing Samples: 10,000

4. Model Architecture

Layer	*Type*	*Neurons*	*Activation*
Input	Dense	784	–
Hidden 1	Dense	128	ReLU
Hidden 2	Dense	64	ReLU
Output	Dense	10	Softmax

5. Experimental Configurations

Weight Initialization Methods:

- Random Normal
- Glorot Uniform (Xavier)
- He Normal

Optimizers Used:

- SGD
- Adam

6. Observations / Results

- Students should include:
- Training loss and accuracy plots
- Tabular comparison of final accuracies
- Epochs required for convergence

7. Result Analysis (Example)

- Random initialization shows unstable training.
- Xavier initialization improves stability but performs best with tanh/sigmoid.
- He initialization shows faster convergence with ReLU.
- Adam optimizer converges faster and more smoothly than SGD.

8. Conclusion(Example)

He initialization combined with Adam optimizer provides the best training stability and convergence speed for deep neural networks using ReLU activation.

Experiment 2: Overfitting Analysis and Regularization Techniques

Mapped to: DL Theory – Unit I

Objective: To study the effect of **Dropout, L2 Regularization, and Batch Normalization** on overfitting in deep neural networks.

Steps

1. Train a baseline MLP/CNN on MNIST.
2. Observe training vs validation loss.
3. Apply Dropout.
4. Apply L2 regularization.
5. Apply Batch Normalization.
6. Compare performance.

Outcome

Students understand how regularization improves generalization.

Lab Record

- Model architectures
- Loss/accuracy plots
- Comparative table
- Conclusion

Experiment 3: Learning Rate Scheduling and Early Stopping

Mapped to: Unit I

Objective: To analyze the impact of adaptive learning rates and early stopping.

Steps

1. Train deep model with fixed learning rate.
2. Apply learning rate decay strategies.
3. Enable early stopping.
4. Compare convergence speed.

Outcome

Students learn training control techniques.

Lab Record

- Learning rate graphs
- Epoch comparison
- Best configuration

Experiment 4: CNN for Image Classification

Mapped to: Unit II

Objective: To implement a CNN for image classification.

Steps

1. Load CIFAR-10 dataset.
2. Design CNN architecture.
3. Train and evaluate model.
4. Visualize feature maps.

Outcome

Students understand spatial feature extraction.

Lab Record

- CNN architecture diagram
- Accuracy results
- Feature maps

Experiment 5: Transfer Learning using Pre-trained CNN

Mapped to: Unit II

Objective: To apply transfer learning using ResNet/VGG.

Steps

1. Load pre-trained model.
2. Freeze base layers.
3. Train classifier head.
4. Fine-tune selected layers.

Outcome

Students learn efficient DL training with limited data.

Lab Record

- Frozen vs fine-tuned comparison
- Accuracy improvement

Experiment 6: Sequential Models – RNN, LSTM, GRU

Mapped to: Unit III

Objective: To compare RNN, LSTM, and GRU for sequence modeling.

Steps

1. Load IMDb dataset.
2. Train RNN, LSTM, GRU models.
3. Compare learning curves.

Outcome

Students understand long-term dependency handling.

Lab Record

- Architecture comparison
- Accuracy & convergence analysis

Outcome

Students understand the limitations of vanilla RNNs.

Lab Record

- RNN architecture
- Learning curves
- Performance metrics

PART B: EXPLAINABLE AI & ETHICAL AI LAB

(Exp.No.7-12, aligned with DL Lab syllabus – 01AMP309)

Experiment 7: Interpreting Tree-Based Models

Mapped to: DL Lab Guideline 2

Objective: To interpret decision-making in a Decision Tree model.

Steps

1. Train decision tree on real dataset.
2. Visualize tree structure.
3. Analyze feature importance.

Outcome

Students understand transparent ML models.

Lab Record

- Tree visualization
- Feature importance
- Observations

Experiment 8: Model Explainability using LIME

Mapped to: DL Lab Guideline 3

Objective: To explain predictions of a black-box DL model using LIME.

Steps

1. Train CNN/MLP classifier.
2. Apply LIME to individual predictions.
3. Interpret feature contributions.

Outcome

Students learn local explainability.

Lab Record

- LIME explanations
- Prediction interpretation

Experiment 9: Model Explainability using SHAP

Mapped to: DL Lab Guideline 4

Objective: To analyze global and local explanations using SHAP.

Steps

1. Train DL model.
2. Compute SHAP values.
3. Plot summary and force plots.

Outcome

Students understand consistent explainability.

Lab Record

- SHAP plots
- Feature impact analysis

Experiment 10: CNN Visualization using Grad-CAM

Mapped to: DL Lab Guideline 5

Objective: To visualize important image regions influencing CNN predictions.

Steps

1. Train CNN model.
2. Apply Grad-CAM.
3. Overlay heatmaps on images.

Outcome

Students understand CNN interpretability.

Lab Record

- Grad-CAM images
- Interpretation

Experiment 11: Counterfactual Explanations & Fairness Analysis

Mapped to: DL Lab Guidelines 6 & 7

Objective: To analyze fairness and generate counterfactual explanations.

Steps

1. Train classifier on biased dataset.
2. Compute fairness metrics.
3. Generate counterfactual examples.

Outcome

Students learn bias detection and mitigation.

Lab Record

- Fairness metrics
- Counterfactual examples
- Ethical analysis

Experiment 12: Explainability & Ethics Case Study

Mapped to: DL Lab Guidelines 8–10

Objective: To analyze explainability and ethical concerns in real-world AI systems.

Steps

1. Select real-world biased AI case.
2. Identify ethical issues.
3. Propose fairness strategies.

Outcome

Students develop responsible AI thinking.

Lab Record

- Case description
- Ethical issues
- Proposed solutions